Training and tuning with TorchTitan and Megatron-LM

From a verified baseline to a qualified MoE scaling ladder

> ⓘ Note
>
> **Replication versus port:** to reproduce Marin's experiments directly, start from the pinned Marin JAX/Levanter model, optimizer and data interfaces. TorchTitan/Megatron are alternative implementations requiring parity and new qualification. See Marin's actual optimizer-search code before choosing a port.

Downloaded implementations and the trainer decision

Both source trees are downloaded under the ignored `tmp/` directory, with persistent archives and inspected files under `references/systems/`. The source pins record:

Trainer	Inspected commit	Starting point
TorchTitan	`1d1fa9444d07b79351c9ed660b3ac4a5f52dfef9`	Python recipe functions, Grain input pipeline, PyTorch distributed checkpointing
Megatron-LM	`e0a4bfed7f1e233b857187ceff0104645858d7d1`	GPT pretraining entry point, indexed datasets, Megatron-Core MoE/parallelism
Marin	`1d274b52c636b6f2a9177b1d4c14b2f74795a8a6`	Scientific recipe/data reference; JAX/Levanter and infrastructure need separate qualification

Candidate decision: benchmark Megatron-LM first for the standard MoE control, and TorchTitan as the alternative if model modification and input-pipeline integration are substantially easier. This is our engineering recommendation, not a measured winner. The previous OLMo-core proposal remains a useful reference, not a completed trainer selection. Commit to one implementation after parity, resume, throughput and modification-cost checks. Maintaining two production trainers would consume this allocation's research budget.

Official references: TorchTitan, Megatron-LM. "Supports MoE" does not mean "supports our tied middle-loop architecture." Neither downloaded snapshot has been qualified here for that design.

Build a reproducible software environment

Create separate training and evaluation environments. Pin the source commit, container digest, Python, PyTorch, CUDA/driver, NCCL, Transformer Engine, attention kernel, grouped-GEMM/dispatch libraries and compiler settings. Do not install nightly requirements into the knowledge repo's environment as part of documentation work.

TorchTitan's current README requests a recent PyTorch nightly. Its configuration is now Python functions returning `Trainer.Config`; older TOML examples should not be assumed valid. Megatron's examples can target earlier releases: the inspected Llama README explicitly checks out `core_r0.12.0`, whereas the downloaded tree is a later commit. Use an example matching the selected release or port and validate it; do not combine an old container, new flags and a third release's checkpoint converter.

Record a resolved configuration for every job, including parameter-group LR/decay multipliers, initialization, attention boundary semantics, optimizer numerical conventions and exact dataset manifest. A run name is not sufficient provenance.

Stage 1: smoke tests that do not consume the whole cluster

For TorchTitan, the pinned README and launcher provide a small bundled-data test:

```
cd /mnt/shared/repos/scaling-ladder-knowledge/tmp/torchtitan
NGPU=1 ./run_train.sh
```

This uses the test tokenizer and debug recipe for ten steps; run it only in the qualified GPU environment. It has **not** been executed here. The command's role is basic environment validation, not convergence evidence. The selected verified recipe can then be launched with `MODULE=... CONFIG=...`; for example the upstream `torchtitan_recipes.models.llama3` module contains `llama3_8b`.

The pinned `run_train.sh` launches a **single-node** rendezvous on localhost. A multi-node job needs an explicit shared rendezvous and the cluster scheduler's rank mapping. A schematic 12-node command, executed once per node, is:

```
# Set MASTER_ADDR, NODE_RANK, JOB_ID, RECIPE_MODULE and RECIPE_NAME in the scheduler.
torchrun --nnodes=12 --nproc_per_node=8 \
  --node_rank="$NODE_RANK" --rdzv_backend=c10d \
  --rdzv_endpoint="$MASTER_ADDR:29500" --rdzv_id="$JOB_ID" \
  -m torchtitan.train --module "$RECIPE_MODULE" --config "$RECIPE_NAME"
```

This is a launch template requiring an authored/validated recipe, scheduler allocation and consistent shared paths; it is not a ready hero launcher. Do not pass an authentication secret on a traced shell command line: the upstream wrapper enables shell tracing.

For Megatron, inspect `pretrain_gpt.py` and the pinned `examples/mixtral/train_mixtral_8x7b_distributed.sh`. First make a small dense configuration and a tiny MoE configuration on 1–8 GPUs, using real local input and strict save/resume tests. The upstream Mixtral script is an illustrative architecture recipe, **not** a 96-GPU configuration: it uses PP4/EP8, and includes `--no-load-optim` and `--no-load-rng`. Those switches must not survive into fault-tolerant continuation of our hero run.

Stage 2: ingest our corpus correctly

Megatron indexed data

The inspected `tools/preprocess_data.py` accepts JSONL, JSON keys, a tokenizer, an output prefix and workers. A preparation template is:

```
cd /mnt/shared/repos/scaling-ladder-knowledge/tmp/megatron-lm
python tools/preprocess_data.py \
  --input /datasets/release-v1/high-web.jsonl \
  --json-keys text \
  --tokenizer-type HuggingFaceTokenizer \
  --tokenizer-model /datasets/tokenizer-pinned \
  --append-eod \
  --output-prefix /datasets/release-v1/megatron/high-web \
  --workers 32
```

These paths are placeholders and the command has not been run. Process **each frozen split and bucket separately**, then preserve all `.bin/.idx` outputs, counts and checksums. Validate tokenizer special-token behavior: appending EOD on top of an automatically inserted EOS can double the separator. The parser's tokenizer options and local tokenizer assets need a hand-checked round trip.

Configure the data blend explicitly over those prefixes and prevent train/validation overlap. Megatron's source-level weights should not be assumed to implement a desired document-probability or token-probability scheme without inspection. Audit realized token shares on a real sample. Distinct lengths, packing and oversampling can change observed exposure.

TorchTitan Grain data

The pinned `torchtitan/components/data/README.md` describes sources, processing, composition/packing, and a checkpointable `GrainDataLoader`. It provides both an `IndexedJsonlSource` and an example `PretokenizedMemmapSource` with token arrays and document offsets. Prefer a prepared local token store for the hero run; downloading from the Hub on each rank risks throttling and makes throughput depend on an external service.

Connect our frozen bucket streams through an explicitly controlled mixture, preserve IDs/offsets for audits, and validate the data-loader state after restart. Default recipes use C4 and may disable checkpointing/validation; replace those choices deliberately. For packed samples, verify segment masks and position resets rather than assuming concatenated documents are isolated.

Stage 3: choose a parallel layout using measured memory and speed

Assume 12 nodes × 8 H100 SXM 80GB, NVLink within nodes and fast inter-node networking; this topology remains to be confirmed. Begin a MoE systems sweep with TP1, PP1 and EP8 within each node. There are twelve expert-replica groups across nodes, but **the dense data-parallel token-stream degree is not twelve**. Expert ownership does not automatically divide the number of independent training sequences by eight.

For a conventional TP/PP/CP layout,

$$B_{\text{tok}} = n_{\text{data}}\, m\, a\, \ell,$$

where n_{data} counts distinct data streams after model/context sharding, m is sequences per data rank per microbatch, a accumulation steps and ℓ sequence length. With TP=PP=CP=1, $n_{\text{data}} = 96, m = 1, a = 8, \ell = 4096$, the batch is 3,145,728 tokens. Confirm the backend's group construction and loss reduction; do not infer it solely from the EP label.

Try EP1/4/8 as memory permits, compare grouped GEMMs and dispatch choices, and measure input stalls and communication. Attention/embedding parameters and expert parameters have different replication groups. Record peak allocated/reserved HBM, model tokens/s, loss-bearing tokens/s, p50/p95 step time, all-to-all time, recomputation and checkpoint costs. A high kernel TFLOP rate with input stalls is not hero throughput.

Do not directly multiply an FP8 dense throughput table by our BF16 MoE duration. Qualify BF16 first; precision changes require their own stability and parity checks. Do not use H100 2:4 sparse peak as if token routing receives that hardware benefit.

Stage 4: freeze the reference training semantics

Start from a standard non-looped decoder/MoE control with RMSNorm, SwiGLU, RoPE, QK normalization and declared router auxiliary losses. OLMoE provides an empirical reference, while Step Law and Power Lines motivate horizon-aware tuning; see the evidence audit.

The frozen recipe must specify:

- Parameter counting, embeddings/head tying, attention heads/KV heads and vocabulary padding.
- BF16 compute versus FP32 master weights, moments, accumulation and reductions; loss scaling and clipping scope.
- AdamW's epsilon location, bias correction, beta values, excluded decay groups and schedule units.
- Router precision, top-k normalization, balancing/z-loss coefficients, layer/microbatch reductions, capacity and dropped-token behavior.
- EOS, position resets, padding/loss masks, and whether attention crosses document boundaries.
- Initialization variance and residual scaling; width/depth parameterization and actual LR groups.

These are scientific controls. Two trainers with the same headline architecture can implement different optimization problems.

Stage 5: a bounded tuning procedure

Use the frozen corpus, evaluation suite and architecture family. A practical first search at the reference size is a **3×3 LR/batch grid**, followed by a decay/timescale bracket at the best neighborhood. Values should bracket stable optima, not assert that a published LR coefficient transfers. Tune at least a second token horizon. Allow expansion when the best run lies on a boundary; budget the expansion before launching.

For a paired recipe comparison, use the same token stream seeds and equal **token horizons**. For a runtime comparison, additionally report tokens/s and wall time. A larger batch changes the number of updates, optimizer timescales and warmup exposure, so count all schedules in tokens as well as steps.

Fit candidate relationships such as

$$\log \eta_\star = a + b \log N + c \log D, \qquad \log B_\star = d + e \log N + f \log D.$$

Also test whether a simpler $B(D)$ fits held-out results adequately. Fit only after inspecting plateaus and stable runs; uncertainty in a broad optimum should not be presented as an exact label. For AdamW, include decay/timescale transfer. For AdamH or Muon, derive a separate law and group definitions rather than reusing AdamW coefficients.

Use fresh initialization for independent trials. Shared WSD prefixes are efficient for endpoint branches if warmup/stable states and data prefixes truly coincide; changing peak LR, batch, decay or optimizer usually prevents prefix sharing. Branch costs and shared training history must be recorded.

At each chosen size, train several horizons near 100/200/400 TPP where feasible, plus short stability screens. Add fresh long-horizon holdouts at the intended 200+ TPP regime. Keep width-only and depth-changing checks distinct. For MoE, hold expert count/top-k fixed in the first transfer family; test expert-count changes separately before calling their transfer validated.

Stage 6: only then add the tied middle loop

A qualified extension needs all of the following:

1. One parameter set for each physical middle block, invoked twice; shared norms/router and no accidentally cloned expert weights.
2. Gradient accumulation from both visits and correct activation recomputation; compare against a tiny unrolled reference.
3. Residual-write scaling for **both** attention and MoE branches; SMELT's middle-half/two-visit setup uses 1/2 for those repeated writes.
4. Explicit auxiliary-loss reduction across visits. Summing twice changes the effective coefficient relative to averaging.
5. Expert parallel dispatch, compiler/checkpoint wrappers and state-dict paths that tolerate repeated module calls.
6. Generation caches keyed by executed visit, with full-forward versus incremental decode parity.
7. Export metadata that describes recurrence; duplicating shared tensors into independent layers changes the model semantics if training/export code treats them differently.

The SMELT guide explains the evidence and limitations. This extension remains unimplemented/unqualified in this knowledge repo. If it fails the time or stability gate, the already tuned standard MoE control is the fallback.

Stage 7: long-run operations and release gates

Proposed qualification: a short single-device numerical reference; 8-GPU collective/EP and save/resume tests; a multi-node real-data test; then a 96-GPU rehearsal long enough to expose recompilation, memory growth, dataloader exhaustion, checkpoint writes and communication skew. Advance only when loss/gradient/token-order checks pass and measured throughput supports the chosen horizon.

Restore model, optimizer, scheduler, RNG, consumed tokens, sampler cursors and mixture-phase state. Compare an interrupted run against uninterrupted continuation on the same topology. A topology-change restart needs separate qualification; some routing/collective orders may lose bitwise equality, but exposure and numerical differences must stay bounded and explained.

Save recoverable checkpoints on a measured time interval plus token milestones; retain immutable complete checkpoints before evaluation. Alert on NaNs, clipping frequency, per-layer norm drift, router collapse, token drops, HBM growth and step-time regressions. A rescue LR or changed data mixture creates a new run segment and invalidates a pre-registered recipe forecast unless separately modeled.

No environment installation, distributed training, GPU tuning or performance validation has been performed in this turn. The downloaded code and command templates make the next engineering work inspectable; they are not an execution claim.